AssetFlow

Smart Asset Management & Resource Allocation Platform

Design Document · v1.0

Cultural Council · IIT Roorkee · 2026

Field	Value
Document Type	Technical Design Document
Project	AssetFlow — Asset Management Platform
Organization	Cultural Council, IIT Roorkee
Version	1.0 — June 2026
Stack	Python / Flask + React 18 + SQLite3
Deliverable	Problem Statement 1 — Cult Open Projects 2026

Section 1

Problem Understanding

The Cultural Council of IIT Roorkee manages a large inventory of shared resources — DSLR cameras, studio lighting rigs, audio consoles, costumes, stage props, and event infrastructure — across dozens of sections and hundreds of annual events. Resource allocation is currently fragmented across WhatsApp groups, spreadsheets, and informal verbal agreements, resulting in double-bookings, untracked losses, inventory blind-spots, and disputes over availability.

AssetFlow replaces this fragmentation with a centralised, role-aware platform where any section member can browse available stock, submit a dated booking request with stated purpose, and track the lifecycle of that request — from pending through approved, issued, and returned — while administrators maintain full operational visibility through live dashboards and an immutable audit trail.

Key Pain Points Addressed

Scheduling conflicts: System prevents bookings that exceed available stock at submission time.

Inventory blind-spots: Live available_quantity counter reflects all issued and approved bookings.

Accountability gaps: Every state transition is audit-logged with user, timestamp, and IP.

Overdue returns: Automatic overdue detection flags items past their due date.

No utilisation data: Analytics dashboard surfaces category-wise utilisation and booking trends.

Section 2

System Architecture

AssetFlow uses a clean two-tier client–server architecture with a stateless REST API backend and a single-page React frontend. The two tiers communicate exclusively over HTTP/JSON; no server-side rendering is involved.

Layer	Technology	Responsibility
Presentation	React 18 + React Router v6	SPA UI, client-side routing, JWT storage
API Gateway	Flask 3 + Flask-CORS	REST endpoints, CORS, request validation
Auth	Flask-JWT-Extended + bcrypt	Token issuance, verification, role claims
Business Logic	Python 3 route handlers	Inventory maths, booking lifecycle rules
Persistence	SQLite3 (stdlib)	Relational storage, WAL mode, FK constraints
QR Module	qrcode + Pillow	Server-side PNG generation, base64 response

Request Flow

1. Browser sends HTTP request with Authorization: Bearer
2. Flask-JWT-Extended validates token; role claim attached to request context
3. @admin_required / @jwt_required decorator enforces access control
4. Route handler queries SQLite, applies business rules, mutates state
5. Audit log entry written in the same DB transaction
6. JSON response returned; React updates UI state via Axios interceptor

Section 3

Database Schema

Five tables form the relational core. SQLite3 enforces foreign-key constraints (PRAGMA foreign_keys = ON) and uses WAL journaling for write concurrency.

Table	Purpose
users	Stores all accounts. role column enforces admin/user split.
assets	Master inventory. available_quantity is the live "free stock" counter.
bookings	Every request, approval, issuance and return. status FSM enforced in Python.
audit_logs	Append-only event log. Never updated after insert.
maintenance_records	Health reports per asset. High-severity auto-flags asset status.

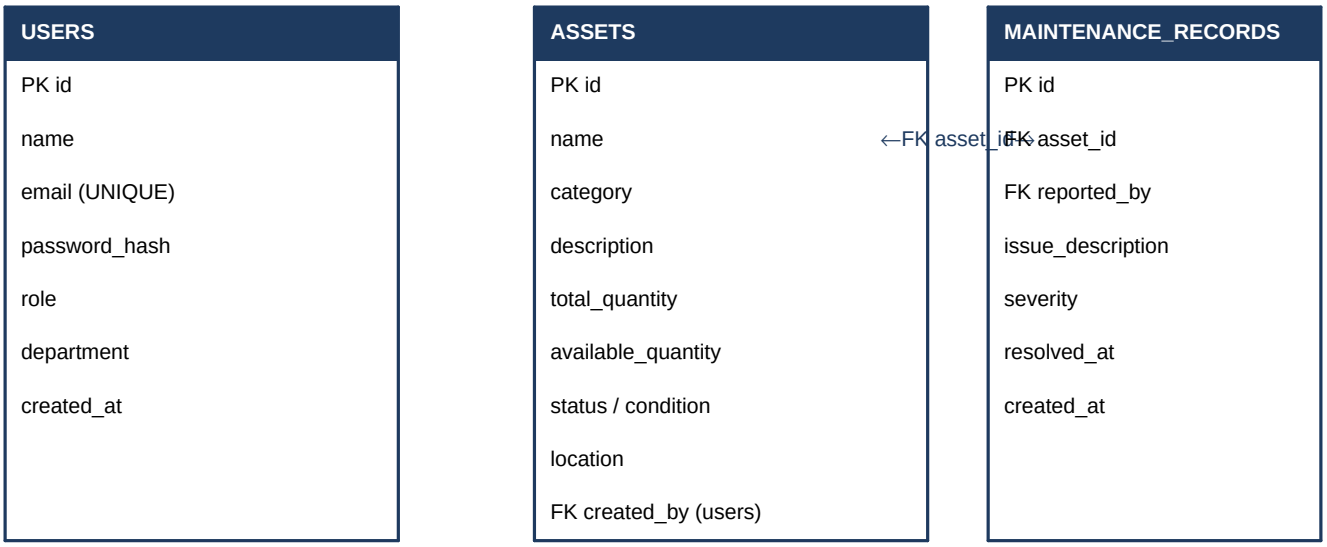
Key Column Definitions

Table.Column	Type	Constraint / Notes
users.role	TEXT	CHECK IN ('admin','user')
assets.status	TEXT	CHECK IN ('available','maintenance','retired')
assets.condition	TEXT	CHECK IN ('excellent','good','fair','poor')
assets.available_quantity	INT	Decremented on issue; incremented on return
bookings.status	TEXT	FSM: pending→approved/rejected, approved→issued, issued→returned/overdue
bookings.due_date	TEXT	Set to end_date at issue time; drives overdue detection
audit_logs.*	—	Append-only; no UPDATE or DELETE ever issued on this table

Section 4

Entity Relationship Diagram (ERD)

The diagram below represents all five entities and their relationships. PK = Primary Key, FK = Foreign Key.



Central BOOKINGS table — links all entities:

BOOKINGS

PK id

FK user_id → users.id

FK asset_id → assets.id

quantity, purpose

start_date, end_date, due_date

status (FSM)

FK reviewed_by → users.id

issued_at, returned_at

admin_note

AUDIT_LOGS (append-only)

```
PK id  |  FK user_id → users.id

action (e.g. BOOKING_APPROVED)  |  entity_type  |  entity_id

details  |  ip_address  |  created_at
```

Section 5

API Overview

All endpoints are prefixed with /api/. Authentication uses Bearer tokens in the Authorization header. Responses are always JSON.

Method	Endpoint	Auth	Description
POST	/auth/register	Public	Create new user account
POST	/auth/login	Public	Authenticate; returns JWT
GET	/auth/me	Any	Get current user profile
PUT	/auth/profile	Any	Update name/dept/password
GET	/assets	Any	List assets (search/filter)
GET	/assets/categories	Any	Distinct category list
GET	/assets/:id	Any	Single asset + maintenance log
POST	/assets	Admin	Create asset
PUT	/assets/:id	Admin	Update asset fields
DELETE	/assets/:id	Admin	Delete (no active bookings)
GET	/assets/:id/qrcode	Any	Generate QR PNG (base64)
POST	/assets/:id/maintenance	Any	Report maintenance issue
GET	/bookings	Any	List bookings (own or all)
POST	/bookings	Any	Submit booking request
PUT	/bookings/:id/approve	Admin	Approve pending booking
PUT	/bookings/:id/reject	Admin	Reject booking
PUT	/bookings/:id/issue	Admin	Issue asset; decrement stock
PUT	/bookings/:id/return	Admin	Record return; restore stock
PUT	/bookings/:id/cancel	Owner/Admin	Cancel pending/approved
POST	/bookings/mark-overdue	Admin	Batch-flag overdue records
GET	/analytics/dashboard	Admin	KPI summary cards
GET	/analytics/utilization	Admin	Asset utilisation rates
GET	/analytics/category-breakdown	Admin	Bookings by category
GET	/analytics/booking-trend	Admin	30-day daily booking count
GET	/analytics/recent-activity	Admin	Last 20 audit events
GET	/analytics/overdue	Admin	Overdue items with users
GET	/analytics/audit-logs	Admin	Paginated full audit trail
GET	/users	Admin	All users with booking counts

GET	/health	Public	Service health check

Section 6

Design Decisions

Python + Flask over Node.js

The container environment does not support native Node addons (node-gyp / better-sqlite3). Python's stdlib sqlite3 module requires zero compilation, providing identical SQL semantics with full WAL-mode and foreign-key support. Flask's minimal footprint keeps the codebase readable and the route logic transparent.

SQLite for persistence

Campus-scale usage (hundreds of assets, tens of concurrent users) sits well within SQLite's write throughput. SQLite eliminates a separate database process, simplifying deployment to a single "python app.py" command — critical for a hackathon submission that must be reproducible.

JWT stateless authentication

Stateless tokens allow the React frontend and Flask backend to be deployed on separate origins without shared session storage. The role claim embedded in the token drives both frontend route guards and backend @admin_required decorators from a single source of truth.

Inventory updated at issue, not at approval

Available quantity is only decremented when an admin physically issues the asset, not at approval time. This mirrors the real Council workflow: approval is a green-light, but the counter should not drop until the item leaves the store. It also allows cancellations after approval without requiring an inventory correction.

Append-only audit_logs

The audit table is never updated or deleted — only inserted into. This provides a tamper-evident activity trail. Every significant state change (asset CRUD, booking transitions) calls the shared audit() helper within the same database connection, guaranteeing log consistency.

QR codes generated server-side

Server-side generation (qrcode + Pillow) produces a consistent PNG regardless of the client browser or OS. The image is returned as a base64 data-URI so no file system storage is needed and the endpoint can be called on demand. The QR payload is a small JSON object with asset id, name, and category — sufficient for scanning during physical issue/return.

React SPA with proxy

The frontend package.json proxy field forwards all /api/* calls to the Flask server during development, eliminating CORS complexity locally. In production, a reverse proxy (nginx) would serve the built React bundle and proxy API calls identically.

Minimal external dependencies

The frontend uses only Recharts for charts and React Router for navigation — no heavy UI framework. All styling is custom CSS with CSS variables, keeping the bundle small and the visual design fully in the team's control.